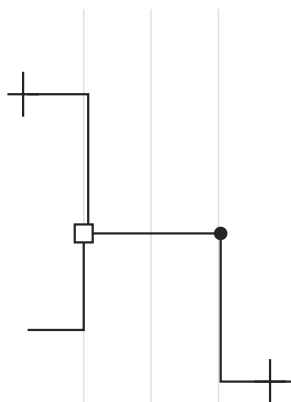


10



CHAPTER / 10

The Call You Didn't Make

IT DEPENDS

TypeScript Principles, Debated

[infer]

01 The Principle

Eden Legacy: "Why does tracking a parcel need a second argument now?"

Idan Greenfield: "It doesn't. The options are for the event history."

Eden: "Tell that to the tracking page."

The page is one of forty callers that reach the courier SDK through the team's integration module. The module keeps the raw client private. It exports `courier`, built by Idan's safe helper, which wraps every method on the client so that each call resolves to a `Result` instead of throwing or rejecting.

Eden opens the module's result type and the tracking-page code that worked with version four:

TYPESCRIPT

01

```
type Result<T> =  
  | { ok: true; value: T }  
  | { ok: false; error: unknown };  
  
async function parcelStatus(code: string): Promise<string> {  
  const found = await courier.retrieve(code);  
  if (!found.ok) return "Tracking is unavailable. Try again  
  shortly.";  
  return found.value.status;  
}
```

Idan: "I didn't want every caller writing its own `try/catch`."

Eden: "And the types for a dozen wrapped methods?"

Idan: "The SDK already declares their arguments and return types. I want our wrapper to derive its signatures from those declarations. Change a request in the SDK, and our types should follow."

Eden: "They followed it this morning. The page stopped compiling."

He opens the v5 declarations. This upgrade adds an overload for retrieving a shipment's event history, and a local tracking-code check:

TYPESCRIPT

02

```
interface Shipment {
  code: string;
  status: string;
}

interface ShipmentWithEvents extends Shipment {
  events: string[];
}

interface Courier {
  readonly region: string;
  retrieve(code: string): Promise<Shipment>;
  // Added in v5.
  retrieve(
    code: string,
    options: { events: true },
  ): Promise<ShipmentWithEvents>;
  // Added in v5.
  isTrackingCode(text: string): boolean;
}

declare const raw: Courier;
const courier = safe(raw);
```

Eden puts a direct SDK call next to the page's call in a scratch file.

TYPESCRIPT

03

```
raw.retrieve("TRK-42"); // Accepted.
courier.retrieve("TRK-42"); // Error: Expected 2 arguments.
```

Idan: "Same function underneath."

Daniel Compiler: "Different type on top. `safe` declares its return type as `Safe<C>`. Here `C` is `Courier`. Open that alias."

Eden: "That's where I stopped. I can follow the mapped keys. There are two `inferred` in the next line."

Daniel brings back his `Unwrap` helper.

TYPESCRIPT

04

```
type Unwrap<T> = T extends Promise<infer U> ? U : T;
```

```
type Parcel = Unwrap<Promise<Shipment>>; // Shipment
```

```
type Code = Unwrap<string>; // string
```

Daniel: "`infer U` introduces a name inside the pattern we're matching. For `Promise<Shipment>`, the match gives `U` the type `Shipment`. We can use that name in the true branch. For `string`, the pattern doesn't match, so we take the other branch and return `T`."

Eden: "And nobody supplies `U` when they use the alias."

Daniel: "Only `T`. The match supplies `U`. Nothing here opens a promise at runtime. Your parcel is still in transit."

The `infer` keyword declares that variable in a conditional type's `extends` pattern. The variable is available in its true branch, not its false branch or outside the conditional.

02 The Debate

"Which call did it look at?"

Idan opens the type that describes the wrapped client.

TYPESCRIPT

05

```
type Safe<C> = {  
  [K in keyof C]: C[K] extends (...args: infer A) =>  
    Promise<infer R>  
    ? (...args: A) => Promise<Result<R>>  
    : C[K];  
};
```

Idan: "For every property, check whether it's a function returning a promise. A is its parameter tuple. R is the value inside that promise. Rebuild the function with the same parameters, returning a promise of our result. Leave the other properties alone."

Eden: "So before this upgrade, retrieve gave it [code: string] and Shipment. The wrapped method was (code: string) => Promise<Result<Shipment>>. That's what the page was calling."

Idan: "I wrote it for our generated tracking client first. Every operation there returns a promise. The courier used to fit too."

Daniel: "Now inspect these."

TYPESCRIPT

06

```
type RetrieveArguments = Parameters<Courier["retrieve"]>;  
// [code: string, options: { events: true }]  
  
type RetrieveReturn = ReturnType<Courier["retrieve"]>;  
// Promise<ShipmentWithEvents>
```

Eden: "Only the second overload? I expected both parameter lists."

Daniel: "These utilities extract from the last call signature. Your pattern does too. `ReturnType` captures the function's return type. Your `Promise<infer R>` goes one step further and captures the value inside it."

Idan: "The direct call has a code and no options. It picks the first overload. Why can't the wrapped call do that?"

Daniel: "By then you've built a function type with one signature. It requires the options. Extraction happened on `Courier["retrieve"]`, with no call arguments to choose from."

Eden scrolls back to the two SDK declarations.

Eden: "Could they publish a final signature with optional options?"

Daniel: "They could. Extraction would then see that signature and its return type. It still wouldn't preserve the two cases for you. The last-signature rule doesn't promise that the last signature covers everything."

Idan: "We can preserve the cases ourselves. They only added one overload."

She gives the integration its own public type and replaces its earlier courier declaration.

TYPESCRIPT

07

```
type SafeCourier = Omit<Safe<Courier>, "retrieve"> & {
  retrieve(code: string): Promise<Result<Shipment>>;
  retrieve(
    code: string,
    options: { events: true },
  ): Promise<Result<ShipmentWithEvents>>;
};

const courier = safe(raw) as SafeCourier;
```

TYPESCRIPT

08

```
courier.retrieve("TRK-42");  
courier.retrieve("TRK-42", { events: true });
```

Eden: "Now we maintain their overloads here too."

Idan: "For this method. The other request signatures still come from the SDK. And the build told us this one needed attention."

Noam Kiperman: "The assertion didn't tell us anything. Show me what it promises."

Idan: "It forwards the arguments, waits for the answer, and catches the error. Both overloads use that same behavior."

She puts the implementation beside the type. This SDK has a flat set of required ordinary methods and data properties.

TYPESCRIPT

09

```
function safe<C extends object>(client: C): Safe<C> {  
  return new Proxy(client, {  
    get(target, key) {  
      const member: unknown = Reflect.get(target, key, target);  
      if (typeof member !== "function") return member;  
  
      return async (...args: unknown[]) => {  
        try {  
          const value: unknown = await Reflect.apply(member,  
            target, args);  
          return { ok: true, value };  
        } catch (error) {  
          return { ok: false, error };  
        }  
      };  
    },  
  }) as Safe<C>;  
}
```

Noam: "The compiler isn't checking that every transformed signature agrees with that function. You're asserting the relationship."

Idan: "I know. We test the wrapper. The raw client isn't exported, so callers can't accidentally skip it."

Eden: "I can keep a raw client private without a Proxy. I did that in the old payments adapter. People added the forwarding methods there because it was the only module they could import."

Idan: "Then show me that version after we've fixed this one. I want to know whether the fix is wrong or you dislike maintaining it."

"You left this one alone?"

Noam selects the false branch of Safe.

Noam: "What reaches : C[K]?"

Idan: "The region string. In version four all the operations returned promises."

Noam: "And this new method?"

Version five also adds `isTrackingCode`. It is a local format check: TRK-42 is valid, bad is not. Eden has used it in the tracking form's new guard.

TYPESCRIPT

10

```
function acceptTrackingCode(text: string): "accepted" |
  "rejected" {
  if (!courier.isTrackingCode(text)) return "rejected";
  return "accepted";
}
```

Eden: "That one compiles. It returns a boolean."

Noam: "The declaration does. Run it with bad."

TYPESCRIPT

11

```
raw.isTrackingCode("bad"); // false
acceptTrackingCode("bad"); // "accepted"
```

Eden: "Oh. It's already wrapped."

Idan: "The Proxy wraps every function. The type only wraps functions returning promises."

Noam: "So the boolean in the editor is a promise at runtime. It's truthy even when it eventually contains `false`."

Eden: "And it resolves to success. `ok: true, value: false`. The SDK ran perfectly."

Idan replaces the mapped type.

TYPESCRIPT

12

```
// Replacement for Safe.
type Safe<C> = {
  [K in keyof C]: C[K] extends (...args: infer A) => infer R
    ? (...args: A) => Promise<Result<Awaited<R>>>
      : C[K];
};
```

Idan: "Now all this client's methods match. Capture the whole return type as `R`. Then `Awaited<R>` describes the value after our `await`. That includes a plain boolean. The region string still takes the false branch."

Noam: "Does the form go red?"

Idan: "The hover says `Promise<Result<boolean>>` now. But this negated call still compiles."

Daniel: "On 7.0.2 I get a diagnostic for `if (promise)`. Put the `!` back and it compiles."

Noam: "Then we're still fixing the condition ourselves. It always gets a promise, whether the editor complains or not."

Eden changes the guard.

TYPESCRIPT

13

```
// Replacement for acceptTrackingCode with the always-async wrapper.
async function acceptTrackingCode(
  text: string,
): Promise<"accepted" | "rejected"> {
  const check = await courier.isTrackingCode(text);
  if (!check.ok || !check.value) return "rejected";
  return "accepted";
}
```

Noam: "Keep bad in the test. Checking only ok would let it through again."

Idan: "And a wrapper test for synchronous methods. I said the build would tell us. It told us about the overload and believed us about this."

Eden: "Why Awaited instead of the Unwrap we already had?"

TYPESCRIPT

14

```
type OneLayer = Unwrap<Promise<Promise<Shipment>>>;
// Promise<Shipment>

type Finished = Awaited<Promise<Promise<Shipment>>>;
// Shipment

type LocalCheck = Awaited<boolean>;
// boolean
```

Idan: "Our Unwrap removes one Promise layer. await keeps resolving. The built-in Awaited models that, including promise-like values. I want our return type to follow the operation we're actually doing."

Noam: "The old fallback wasn't wrong for Unwrap. Keeping a string as a string was the point. It was wrong here because the wrapper changed a function that the type said it would leave alone."

"We're awaiting a format check"

Eden puts the new guard beside the original.

Eden: "This works. We also made a local format check asynchronous."

Idan: "We can leave that method synchronous. It's documented to return a boolean for every string. There isn't an SDK request to recover from."

Sahar Firstclass: "Where would that exception go?"

Idan: "In the courier wrapper. In both its type and its implementation."

She tries a courier-specific version. It uses the repaired `SafeCourier` for requests and preserves the original check signature.

TYPESCRIPT

15

```

type CourierWithSyncCheck = Omit<SafeCourier, "isTrackingCode">
  &
  Pick<Courier, "isTrackingCode">;

function safeCourier(client: Courier): CourierWithSyncCheck {
  return new Proxy(client, {
    get(target, key) {
      const member: unknown = Reflect.get(target, key, target);
      if (typeof member  $\equiv$  "function") return member;
      if (key  $\equiv$  "isTrackingCode") return member.bind(target);

      return async (...args: unknown[])  $\Rightarrow$  {
        try {
          const value: unknown = await Reflect.apply(member,
target, args);
          return { ok: true, value };
        } catch (error) {
          return { ok: false, error };
        }
      };
    },
  }) as unknown as CourierWithSyncCheck;
}

```

Idan: "Replace our exported `courier` with `safeCourier(raw)` and the original synchronous guard works. We test that it receives a boolean, not just that the type accepts the condition."

Noam: "as `unknown` as. You'd send that back in a parser review."

Idan: "I would. The constructor still calls this a `Courier`, though we've replaced its methods. The assertion doesn't check that replacement. We have to call both overloads and the synchronous check."

Noam: "And the exception appears in the type and the handler. Keep them tested together."

Idan: "We can make the exclusions one list and derive the type from it
—"

Eden: "Before we build the configurable version, here's the adapter I meant."

His first piece runs an operation and captures its outcome. The operation can return a value or a promise, and can throw before returning either.

TYPESCRIPT

16

```
async function attempt<T>(  
  operation: () => T,  
) : Promise<Result<Awaited<T>>> {  
  try {  
    return { ok: true, value: await operation() };  
  } catch (error) {  
    return { ok: false, error };  
  }  
}
```

Eden: "Pass it a function so the SDK call happens inside the try. This body is checked. No assertion about a Proxy."

Idan: "And the overloads?"

TYPESCRIPT

17

```
function retrieve(code: string): Promise<Result<Shipment>>;
function retrieve(
  code: string,
  options: { events: true },
): Promise<Result<ShipmentWithEvents>>;
function retrieve(code: string, options?: { events: true }) {
  return attempt(() =>
    options ? raw.retrieve(code, options) : raw.retrieve(code),
  );
}

const courierAdapter = {
  region: raw.region,
  retrieve,
  isTrackingCode: (text: string) => raw.isTrackingCode(text),
};
```

Idan: "You copied the overloads too. And now there's a branch."

Eden: "I did. `raw.retrieve(code, options)` wouldn't match either overload when `options` might be undefined. These two calls do."

Idan: "My wrapper forwards whatever arguments arrive. It doesn't need to make that decision again."

Eden: "That's a cost of mine. But the compiler can check these calls against the SDK. If they change, I have a body to fix."

Noam: "Keep the tests for both return shapes. Writing overloads by hand doesn't prove the implementation returns the right shape for each one."

Eden: "Same tests for both versions. And neither exports `raw`."

Sahar: "What do you still get from mapping the whole client?"

Idan: "Coverage. A new request method gets the same result behavior without someone adding a forwarding function. The normal signatures stay attached to the SDK. Your adapter has another list of the operations we expose."

Eden: "A new method isn't exposed until someone adds it. After finding this one, I'm comfortable with that."

Idan: "Another required method returning a boolean would get the right async signature now. We'd have to decide whether it also deserves an exception."

Eden: "Agreed. I'm objecting to having that decision arrive through the wrapper's default."

03 The Verdict

Eden expands the integration module's existing export. The callers import its `courier` object, not the SDK package.

Eden: "I started counting forty call sites. That's the wrong number. We can keep this export and its call signatures. Replacing the implementation means writing the forwarding methods here."

Idan: "A dozen of them."

Eden: "A dozen. More code than your repair. I'd still do it for this client. We've got overloaded requests and a synchronous helper already. I want the next method reviewed where we call it."

Noam: "Either repaired version can pass these tests. For the adapter, keep the raw client private, keep the overload tests, and keep the invalid-code case. Don't replace the wrapper test with 'no assertions found.'"

Idan switches to the generated tracking client, where the wrapper started.

TYPESCRIPT

18

```
interface TrackingApi {
  lookup(request: { code: string }): Promise<Shipment>;
  history(request: { code: string }): Promise<string[]>;
}

type SafeTrackingApi = Safe<TrackingApi>;
```

Idan: "This generator emits one request object and one promise return per operation. No overloads, no generic methods, no optional methods. I want new generated operations to be covered when the schema adds them. And I can stop a synchronous helper at the wrapper's entry point."

She adds a constraint for that client surface.

TYPESCRIPT

19

```
type AsyncMembers<C> = {
  [K in keyof C]: C[K] extends (...args: infer A) => infer R
    ? [R] extends [PromiseLike<unknown>] ? C[K] : never
    : C[K];
};

function safeRequests<C extends AsyncMembers<C>>(client: C):
  Safe<C> {
  return safe(client);
}

declare const trackingApi: TrackingApi;
const tracking = safeRequests(trackingApi); // Accepted.

safeRequests(raw); // Error: Type '(text: string) => boolean' is
  not assignable to type 'never'.
```

Idan: "For a method, infer its return type and require the whole thing to be promise-like. The brackets check the whole return union. If it fails, that property becomes never, which the supplied method can't satisfy. A new required synchronous method stops the build at safeRequests."

Noam: "Would this preserve the courier overloads?"

Idan: "It wouldn't. Extraction still reads the last signature. This checks the return requirement for the simple methods our generator emits. Optional methods would need different handling too."

Eden: "So the courier doesn't pass. The generated client does, and if it grows a synchronous method, we have to deal with it there. I'd take that change."

Idan: "I'll switch tracking to safeRequests. You can replace the courier export."

Eden adds the remaining forwarding methods to the upgrade. Noam runs the invalid-code case again against the courier adapter.

04 Additional Takes

Daniel Compiler: "Overloads aren't the only thing a call can tell you that extraction can't. Try a generic function."

TYPESCRIPT

20

```
function keep<T>(value: T): T {  
  return value;  
}  
  
type Extracted = ReturnType<typeof keep>; // unknown  
  
const preserved = attempt(() => keep({ code: "TRK-42" }));  
// Promise<Result<{ code: string }>>
```

"Without a particular call, there's no particular T to recover here. At the call, the argument supplies one. A wrapper that extracts and rebuilds this signature has work to do if it wants to preserve that relationship."

Noam Kiperman: "All that, and we're still calling it `Safe`."

SOURCES IN THIS CHAPTER

1. `infer`

<https://www.typescriptlang.org/docs/handbook/2/conditional-types.html#inferring-within-conditional-types>

2. `last-signature` rule

<https://www.typescriptlang.org/docs/handbook/release-notes/typescript-2-8.html#type-inference-in-conditional-types>

3. `Awaited`

<https://www.typescriptlang.org/docs/handbook/utility-types.html#awaitedtype>